Ishan Agarwal[1*], Nia Williams[2], Herman A. Thomas[1] and Christian Deloach[1]

# Systematization of Functional Encryption Approaches in Privacy-Preserving Machine Learning: Trends, Frameworks, and Open Challenges

***Corresponding Author:** Ishan Agarwal, Department of Information Security, University of Tokyo, Japan
Email: ishan_aggarwal1276@student.uj.edu
Nia Williams, School of Computer Science, University of Cape Town, South Africa

arXiv:2204.17836v1 [cs.CR] 11 Apr 2024

**Abstract:** With the advent of functional encryption, new possibilities for computation on encrypted data have arisen. Functional Encryption enables data owners to grant third-party access to perform specified computations without disclosing their inputs. It also provides computation results in plain, unlike Fully Homomorphic Encryption.
The ubiquitousness of machine learning has led to the collection of massive private data in the cloud computing environment. This raises potential privacy issues and the need for more private and secure computing solutions. Numerous efforts have been made in privacy preserving machine learning (PPML) to address security and privacy concerns. There are approaches based on fully homomorphic encryption (FHE), secure multiparty computation (SMC), and, more recently, functional encryption (FE). However, FE-based PPML is still in its infancy and has not yet gotten much attention compared to FHE-based PPML approaches. In this paper, we provide a systematization of PPML works based on FE summarizing state-of-the-art in the literature. We focus on Inner-product-FE and Quadratic-FE-based machine learning models for the PPML applications. We analyze the performance and usability of the available FE libraries and their applications to PPML. We also discuss potential directions for FE-based PPML approaches. To the best of our knowledge, this is the first work to systematize FE-based PPML approaches.

**Keywords:** Privacy-preserving Machine Learning, Functional Encryption, Computation on Encrypted Data

## 1 Introduction

Machine learning's expansion into domains such as computer vision, natural language processing, and speech processing has resulted in a plethora of incredible applications that have become an inseparable part of people's lives. More real-world machine learning applications today rely on a cloud computing environment following the concept of machine learning as a service (MLaaS) [45]. More and more highly regulated enterprises and organizations, for example, banks, governments, insurances, and health, are migrating their data and machine learning services to the cloud. As a result of this advancement, there arises an increase in demand for secure and confidential computing solutions that preserve data and model privacy in ML applications relying on cloudbased machine learning. In light of this, researchers are paying close attention to privacy-preserving machine learning (PPML) [31, 46, 51]. The PPML's purpose is to deal with the problems related to data and model privacy during the training to deployment stages of machine learning.
In the literature, fully homomorphic encryption (FHE) [25], secure multi-party computation (SMC) [12], federated learning [40], differential privacy [23], and trusted execution environments [48] are existing approaches to achieve PPML. In addition to this, functional encryption [14] is also advancing day by day. Fully homomorphic encryption is an encryption standard that allows users to compute over encrypted data without decrypting the ciphertext itself. Secure multiparty computation allows many parties to jointly compute a function over their inputs while keeping their inputs private. On the other hand, federated learning is a machine learning strategy that allows multiple clients to train a model owned by a server while keeping the input data of each participant private. Differential privacy enables the quantification of an algorithm's level of privacy protection to the underlying sensitive dataset and facilitates machine learning model training on private data. A secure enclave, also known as a trusted execution environment (TEE), is a combination of software and hardware elements that creates an isolated environment in a system that ensures the security of applications running inside the TEE. Functional encryption is a type of encryption comparable to homomorphic encryption in which computations can be performed on encrypted data. The key distinction between homomorphic and functional encryption is that homomorphic encryption produces ciphertext results, whereas functional encryption produces plaintext results.

There are primarily two ways to achieve PPML: Fully Homomorphic Encryption [25] and Secure Multiparty Computation [12]. Dowlin et al. proposed a method for transforming a trained neural network to a CryptoNet [26], that enables data owners to send homomorphically encrypted data to the server and receive an encrypted inference. In another approach called CryptoDL, Hesamifard et al. [30] proposed an FHE-based method for privacy-preserving inference on pre-trained convolutional neural networks. Al-Badawi et al. developed Private Fast Text (PrivFT)[9], a method that utilizes FHE for privacy-preserving text classification. Li et al. introduced HomoPAI [35], an FHE-based machine learning platform built on top of PAI (Platform of Artificial Intelligence, an Alibaba Cloud product), enabling data owners from different organizations to store their encrypted data and perform collaborative machine learning. Jäschke et al. in [52] proposed an approach to perform unsupervised machine learning where they implemented a K-means clustering algorithm using HE.

ML Confidential [27] proposed a binary classification method using polynomial approximations and HE. In SecureML [42], Zhang et al. proposed an efficient twoparty protocol for training linear regression, logistic regression, and neural networks models. Rouhani et al. in DeepSecure [46] proposed an approach for the scalable execution of deep learning models in the PPML setting that uses an optimized version of Yao's Garbled Circuit (GC) protocol. Wagh et al. in SecureNN [51] proposed a three-party computation protocol for privacy-preserving training and inference on convolutional neural networks.

In this paper, we consider PPML approaches using functional encryption. When machine learning is offered as a service, the model is stored on the server, and one or more clients are responsible for training. Sometimes there might be pre-trained models possessed by the server. In general, FHE-based machine learning models are trained on unencrypted data, and then their inference is obtained on encrypted data. In the FHE-based machine learning setting, a client sends encrypted data to the server; the server then performs tasks like classification using the pre-trained ML model on the data received from the client and produces the prediction results in the ciphertext. Even though the server performs computations on encrypted data, it cannot learn anything and generates ciphertext predictions. So, only the owner of the data can see the actual result. However, in the case of functional encryption, the server generates the computation results in plaintext using a special key that enables partial decryption needed for computation (see Fig. 1). The rest of the steps are identical to those of homomorphic encryption. In the realm of PPML, we found the idea of producing plaintext results over encrypted data without fully decrypting vacy protection to the underlying sensitive dataset and facilitates machine learning model training on private data. A secure enclave, also known as a trusted execution environment (TEE), is a combination of software and hardware elements that creates an isolated environment in a system that ensures the security of applications running inside the TEE. Functional encryption is a type of encryption comparable to homomorphic encryption in which computations can be performed on encrypted data. The key distinction between homomorphic and functional encryption

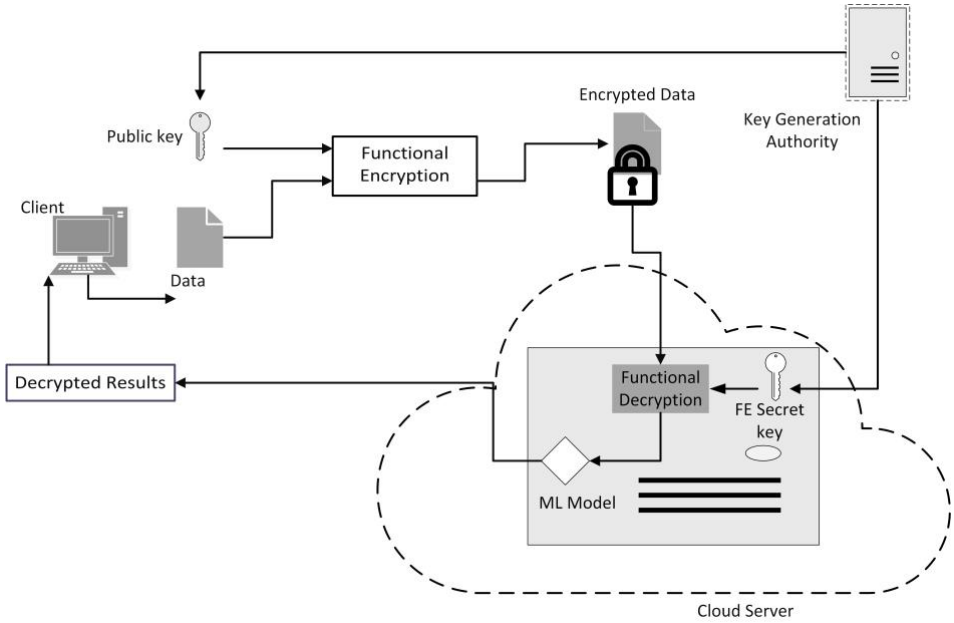


**Fig. 1.** Overview of PPML using Functional Encryption

is that homomorphic encryption produces ciphertext results, whereas functional encryption produces plaintext results.

There are primarily two ways to achieve PPML: Fully Homomorphic Encryption [25] and Secure Multiparty Computation [12]. Dowlin et al. proposed a method for transforming a trained neural network to a CryptoNet [26], that enables data owners to send homomorphically encrypted data to the server and receive an encrypted inference. In another approach called CryptoDL, Hesamifard et al. [30] proposed an FHE-based method for privacy-preserving inference on pre-trained convolutional neural networks. Al-Badawi et al. developed Private Fast Text (PrivFT)[9], a method that utilizes FHE for privacy-preserving text classification. Li et al. introduced HomoPAI [35], an FHE-based machine learning platform built on top of PAI (Platform of Artificial Intelligence, an Alibaba Cloud product), enabling data owners from different organizations to store their encrypted data and perform collaborative machine learning. Jäschke et al. in [52] proposed an approach to perform unsupervised machine learning where they implemented a K-means clustering algorithm using HE.

ML Confidential [27] proposed a binary classification method using polynomial approximations and HE. In SecureML [42], Zhang et al. proposed an efficient twoparty protocol for training linear regression, logistic regression, and neural networks models. Rouhani et al. in DeepSecure [46] proposed an approach for the scalable execution of deep learning models in the PPML setting that uses an optimized version of Yao's Garbled Circuit (GC) protocol. Wagh et al. in SecureNN [51] proposed a three-party computation protocol for privacy-preserving training and inference on convolutional neural networks.

In this paper, we consider PPML approaches using functional encryption. When machine learning is offered as a service, the model is stored on the server, and one or more clients are responsible for training. Sometimes there might be pre-trained models possessed by the server. In general, FHE-based machine learning models are trained on unencrypted data, and then their inference is obtained on encrypted data. In the FHE-based machine learning setting, a client sends encrypted data to the server; the server then performs tasks like classification using the pre-trained ML model on the data received from the client and produces the prediction results in the ciphertext. Even though the server performs computations on encrypted data, it cannot learn anything and generates ciphertext predictions. So, only the owner of the data can see the actual result. However, in the case of functional encryption, the server generates the computation results in plaintext using a special key that enables partial decryption needed for computation (see Fig. 1). The rest of the steps are identical to those of homomorphic encryption. In the realm of PPML, we found the idea of producing plaintext results over encrypted data without fully decrypting it to be intriguing for some of the applications. Note that both HE and FE-based PPML works need high computation costs.

Consider a hospital that outsources the prediction of cardiovascular diseases to a machine learning as a service (MLaaS) company. Assume that this MLaaS company is responsible for performing machine learning operations on sensitive data like patient records. In such a scenario, a good ML model should predict the correct results and should also preserve the privacy of the patients' data. Earlier approaches to such cases can perform training of a machine learning model on the plain data and inference on homomorphically encrypted data. However, when an MLaaS is expected to produce the result in plain HE-based approaches needs an additional key to decrypt the results predicted by the server. Functional encryption can be used to avoid sharing an additional key for the decryption of results inferred by the ML model.

This SoK paper aims to study several research works in the field of functional encryption-based privacy-preserving machine learning. We investigate and examine the impact of functional encryption in the machine learning domain, i.e., machine learning computations on encrypted data. This is intended to give the research community a clear snapshot of FE-based PPML approaches. We only focus on FE-based PPML works out of many other existing PPML approaches.

Initiatives have been taken in recent years to develop FE-based PPML systems. FE-based PPML approaches are brodly categorized as 1) Inner-product FEbased (IPFE) approaches and 2) Quadratic FE-based (QFE) approaches. Ligier et al. [36] proposed an approach to perform privacy-preserving classification on data encrypted using IPFE. An IPFE-based deep neural network approach on MNIST dataset proposed by Xu et al. in [53]. Panzade et al. in [43] proposed a new approach to compute faster secure activation function based on function-hiding inner product encryption for PPML. In [47], Ryffel et al. introduced a system that used QFE with adversarial training to perform privacypreserving predictions. Marc et al. [39] presented the first fully-fledged FE libraries as well as a variety of applications for developing privacy-enhanced machine learning models.

The following are the primary contributions of this paper:

1. We present a basic but substantial theoretical foundation to help researchers understand current approaches to FE-based PPML.
2. We provide a thorough review of the literature on FE-based PPML, emphasizing the strengths and

shortcomings of the various approaches to assessing how they supplement one another.

3. We examine the current constraints that prohibit the implementation of existing FE-based PPML solutions in real-world settings, mostly due to issues with efficiency and usability.
4. We provide research directions to intensify existing works in terms of time performance and security that the research community may pursue in the coming years.

# 2 Background Knowledge

Functional encryption is a generalization of public-key encryption that allows a key holder to compute a particular function of encrypted data using constrained secret keys [14]. Here this function is called functionality. e.g., an FE scheme may be particularly designed to compute inner-products; in this case, the functionality becomes inner-product. In the FE scheme, a key management authority with a master secret key generates a secret key $sk_{fe}$; a decryptor can use that to compute a function on an encrypted message x. This section summarizes the two major functional encryption schemes, inner-product functional encryption, and quadratic functional encryption, used by privacy-preserving machine learning approaches.

## 2.1 Inner-product Functional Encryption

The Decisional Diffie-Hellman assumption (DDH) underpins the method outlined by Abdalla et al. in [2]. Let GroupGenerator be a probabilistic polynomial-time (PPT) algorithm with input security parameter $1^\lambda$, which produces a triplet (G, $p$, $g$), where G is a group of order p created by g *in* G. The tuples (g, $g^a$, $g^b$, $g^{ab}$) and (g, $g^a$, $g^b$, $g^c$) are computationally indistinguishable, according to DDH, where (G, $p$, $g$) ← GroupGenerator($1^\lambda$), and a, b, c $\in Z_p$ are chosen uniformly and independently at random.

The $\Pi_{ipfe}$ = ( Setup, Encrypt, KeyDerivation, Decrypt) FE scheme for IPFE in DDH is as follows: **Setup($1^\lambda$, $1^l$):** This algorithm samples (G,$p$,$g$)← GroupGenerator($1^\lambda$) and s = ($s_1$,.....$s_l$)← $Z^l_p$, sets mpk = ($h_i$ = $g^{s_i}$)$_{i\in[l]}$ and msk = s and finally returns a pair of (mpk, msk).

**Table 1.** Symbols and acronyms used in the paper

| Acronym / symbol | Description |
|---|---|
| mpk | Master Public Key |
| msk | Master Secret Key |
| $sk_{fe}$ | FE Key for IPFE |
| $sk_{qe}$ | FE key for QFE |
| Ct | Ciphertext |
| IND-CPA | security against chosen-plaintext attacks |
| ERT | Extremely Randomized Trees |
| IPFE | Inner-product Functional Encryption |
| QFE | Quadratic Functional Encryption |

**Encrypt(mpk, x):** This algorithm takes mpk and message x = ($x_1$,.....$x_l$) $\in Z^l_p$ as input, chooses random number r ← $Z_p$, computes $Ct_0 = g^r$ and, for each i $\in$ [l], $Ct_i = h_i^r . g^{x_i}$ and returns ciphertext $C_t$.

**KeyDerivation(msk, y):** This algorithm takes msk and vector y=($y_1$,.....$y_l$) $\in Z_p$ as input and outputs key $sk_{fe}$.

**Decrypt(mpk, Ct, $sk_{fe}$):** This algorithm takes the master public key, ciphertext and $sk_{fe}$ for vector y as input and outputs the discrete logarithm in basis g of

$$\prod_{i\in[l]} Ct_{y_i} / Ct_{sk_{0\,fe}}$$

## 2.2 Quadratic Functional Encryption

Quadratic functional encryption scheme uses bilinear groups (also known as pairing groups), has been proposed by [13, 32]. In the case of QFE-based PPML, we refer to schemes proposed by Ryffel et al. in [47]. Here, GroupGenerator is a PPT algorithm on inputting $1^\lambda$ returns $PG = (G_1, G_2, p, g_1, g_2, e)$ of an asymmetric bilinear group, where $G_1$ and $G_2$ are cyclic groups of prime order $p$ (for a $2\lambda$-bit prime $p$ ) and $g_1$ and $g_2$ are generators of $G_1$ and $G_2$, respectively. The application $e : G_1 \times G_2 \rightarrow G_T$ is an admissible pairing i.e. it can be efficiently computable, non-degenerated, and bilinear: $e(g_1^\alpha, g_2^\beta) = e(g_1, g_2)^{\alpha\beta}$ for any scalars $\alpha, \beta \in Z_p$. Therefore, $g_T := e(g_1, g_2)$ which makes the group $G_T$ of order $p$, where $p$ is prime. For any

$$g_{su_n} \in \mathbb{Z}_p^n, v \in \mathbb{Z}_{n_p},$$

Similarly, for any vectors $u$ It is denoted by $e$ $(g_1^u, g_2^v) = \prod_{1-1} e(g_1, g_2)^{\mathbf{u}_t \cdot \bar{v}_1} = e(g_1, g_2)^{\mathbf{u}\cdot\boldsymbol{v}} \in \mathbb{G}_T$, since $\boldsymbol{u} \cdot \boldsymbol{v}$ denotes the inner-product between the vectors $\boldsymbol{u}$ and $v$, that is: $\boldsymbol{u} \cdot \boldsymbol{v} := \sum_{i=1}^{n} u_1 v_i$.

Ryffel et al. in [47] build an efficient FE scheme shown below for the set of functions defined, for all $n, B_x, B_y, B_f \in N^*$, as $F_{n,B_x,B_y,B_f} = f : [-B_x, B_x]^n \times [-B_y, B_y]^n \rightarrow Z$ where the functions $f \in F_{n,B_x,B_y,B_f}$ are expressed as a set of bounded coefficients $f_{i,j} \in [-B_f, B_f]_{i,j\in[n]}$, and for all vectors $\boldsymbol{x} \in [-B_x, B_x]^n, \boldsymbol{y} \in [-B_y, B_y]$, :

$$f(\boldsymbol{x},\boldsymbol{y}) = \sum_{i,j\in[n]} f_{i,j} x_i y_j$$

We defer the readers to [2] and [11, 47] for more cryptographic details on IPFE and QFE schemes, respectively.

## 2.3 Neural networks

The artificial neural network, often known as a neural network, is a machine learning model that is hierarchical and non-linear, with several layers and several neurons in each

layer. Each layer of a neural network processes the input provided by the previous layer before passing it on to the next.

- **Input layer:** The preprocessed raw data or features extracted from raw data in a particular format make up the first layer of the neural network.
- **Hidden Layer:** A neural network can have one or more hidden layers. The First Hidden layer's neurons are linked to the input layer and followed by an activation function. Further hidden layers are fed with the previous layer's output. Weight values are associated with layers, and they are updated during the forward and backpropagation processes until convergence.
- **Activation function:** The activation function of a neuron in a neural network determines the output of that neuron given a single or group of inputs. In machine learning, there are several activation functions such as sigmoid, Rectified Linear Unit (ReLU), and tanh. The ReLU activation function is an example of one of the most frequent activation functions. If the input value is less than zero, the ReLU activation function returns zero, and if it is larger than zero, it returns the same input.
- **Output layer:** The output layer of a neural network is the final layer of neurons that provides the network's output.
-

### 2.4 Polynomial Neural Network

Polynomial neural networks are the kinds of neural networks that primarily facilitate linear components like fully connected layers, convolutions with average pooling, and activation functions are approximated using polynomials. They have demonstrated fairly high accuracy for the relatively simple benchmarks in image recognition tasks [8, 26]. They have also been used to propose the applicability of novel machine learning protocols in various early-stage implementations, as presented in [15, 19]. The simplicity of the operations upon which polynomial neural networks are built ensures high efficiency, particularly for gradient computations. Research works such as [37] have demonstrated that they can reach convergence rates comparable to those of networks with non-linear activation functions.

## 3 Scope and Methodology

Our analysis of the FE-based PPML methodologies is divided into two parts. To begin, we conduct a thorough review of existing methodologies and emphasize their salient features and functions. Second, we evaluate these tools in practice by comparing their usability, complexity, and performance across various case study scenarios. We integrate quantitative performance analysis with a qualitative review, highlighting the difficulties inherent in designing various applications. The secure computation ecosystem encompasses a wide variety of tools. On the low level, there exist math libraries that facilitate the construction of FE implementations, for example, by efficiently implementing algorithms useful for generic lattice-based cryptography. Then there are FE libraries that implement certain schemes and provide slightly more advanced APIs, such as keygen, encrypt, key generation, and decrypt. These libraries abstract away computation features like parameter selection, encryption, and decryption by providing a higherlevel language in which developers can implement their computation.

The primary objective of this work is to comprehend the landscape of MLaaS in data-sensitive scenarios via privacy-preserving machine learning computing, for example, when data supplied to third parties for processing is partially decrypted. PPML ensures the privacy and confidentiality of input data. Additionally, they alleviate excessive pressures on the client endpoint in computing as cloud server does most of the computing part. Finally, they can be used in machine learning situations where clients can contribute data toward a training goal. We also examine the adaptation of current machine learning approaches to cryptographic PPML structures based on functional encryption as a secondary goal. PPML is frequently used in conjunction with other approaches. We discuss the application of all PPML approaches based on FE just at these crossing places. Although homomorphic encryption is a widely used technique in PPML, we only include FE-based approaches because of fewer available articles to the research community on this topic. This work does not cover federated learning, differential privacy, and secure multi-party computation due to their different computational infrastructure requirements and implementation.

PPML that is based on FE has efficiency and usability concerns. FE-based PPML is limited in its computing capabilities due to performance concerns associated with extensive computations. Even though they have been frequently used in other contexts, like private data aggregation and statistics, their application to machine learning is not simple. Moreover, the works focusing on FE-based PPML are significantly lower in number. So, we consider a solution efficient if it offers enhancements to previous methods that reduce their runtime on machine learning applications.

The second problem is associated with the techniques' deployability. Numerous frameworks and technologies make advanced machine learning accessible to data scientists who are not necessarily professionals in computer sciences. However, adopting these frameworks for use with PPML is challenging. We consider usability enhancements in this area if the proposal fits one of the following two requirements. To

start, if it makes the solution more adaptable to existing machine learning frameworks (for example, by giving tools that reduce total programming effort). As such, our study incorporates works that propose Application Programming Interfaces (APIs), compilers, or other significant practical tools that aid in the implementation and deployment of theoretical ideas into real applications, hence increasing their usability. Second, when an open-source implementation supplements the concept. Apart from facilitating future revisions to the approach, open-source implementations ensure that the results are reproducible. Additionally, we verify whether the work contains references to open-source implementations or is released independently (for example, by visiting the authors' websites or GitHub repositories).

Furthermore, we investigate and analyze two parameters of such open-source repositories: i) the code's adherence with the theoretical claims (for example, if it employs the security methods and features stated in the paper); and ii) how well the source code is maintained or integrated with other current frameworks (for example, the version updates since it was released).

In summary, we examine the following for each of the proposals utilizing FE-based PPML: (i) the problem being addressed (training, inference, or both), (ii) the machine learning model used, (iii) the specific FE techniques involved (IPFE or QFE), and (iv) the efficiency and usability considerations examined.

We execute searches in multiple scholarly repositories and databases, mainly Google scholar, IEEE Xplore and ACM digital library for relevant publications using specified keywords such as privacy-preserving machine learning, functional encryption, secure computation, and computation over encrypted data. Secondly, we select those focused on FE-based PPML and have been published in renowned venues like NeurIPS, PETS, IEEE S&P, EUROCRYPT, TCC. We then read their abstracts and methodologies to determine if they fall within the topic of our research. This provides us with an initial collection of works, which we meticulously studied. Finally, we use snowball sampling to add additional suitable works by using references from the first collection of papers. We made every effort to include all significant works implementing PPML using FE.

# 4 Overview of FE-based PPML models and libraries

## 4.1 What has been done?

There are two variations of the FE-based PPML methodologies available in the literature. The first one uses inner-product functional encryption, whereas the other one uses quadratic functional encryption. The IPFE-based methodologies involve training and inference, whereas the QFE methodologies involve simply the inference stage of machine learning.

### 4.1.1 Inner-product FE-based Machine Learning

In this type of methodology, the inputs are encrypted using inner-product functional encryption. Then during the activation, inner products between encrypted inputs and weight matrices are unfolded based on the special property of FE. Later, the neural network operations are done similarly to regular neural networks. Here both forward propagation and backpropagation can be made secure using FE. This methodology supports both training and prediction over encrypted data, unlike the quadratic FE-based approaches.

### 4.1.2 Quadratic FE-based Machine Learning

In quadratic FE-based methodologies, the training phase happens similar to the regular neural networks. It is also worthwhile to note that they train the neural networks with plain data. In the prediction phase, encrypted data is fed to the neural network. It undergoes the process of polynomial approximation, and then the other steps are applied similar to regular neural networks. These approaches are found to be faster than inner-product FE-based approaches.

## 4.2 Cryptography libraries for Functional Encryption implementation

Presently, there are only two dedicated libraries that focus on implementing the state-of-the-art FE schemes. The first one is called CiFEr, and the other is GoFE. Three entities are involved in functional encryption and decryption: an encryptor, a decryptor, and a key management authority. An encryptor encrypts the data before sending it to the decryptor as ciphertext. The decryptor decrypts the ciphertext. The key management authority handles the responsibility of generating a variety of cryptographic keys. In the FE scheme, based on the involvement of encryptors, it can be either single input or multi-input. We detailed both of these libraries and supporting cryptography libraries used in the FEbased PPML works as follows:

### 4.2.1 CiFEr

CiFEr [39] is a functional encryption library developed by FENTEC group that is written in the C language. This library provides developers to use functions to perform various FE operations like encrypt, decrypt, and key generation. These libraries are built in such a way that the predefined functions can be directly called without setting many parameters. Here, the cryptographic key generation is a little abstract to the user,

and users can set the security parameter in terms of bits. It provides both single and multi-input FE implementation.

#### 4.2.2 GoFE

GoFE [39] is another library provided by the fentec group that is written in Golang. Like CiFEr, it also provides an option to use FE functions. Both of CiFEr and GoFE have the same set of state-of-the-art functional encryption implementations. Their performance is based on the underlying programming languages.

#### 4.2.3 FLINT

FLINT [29] is a cryptography library that is used for performing number theory-related operations. It is written in C. Unlike GoFE and CiFEr, it does not dedicatedly provide FE functionality. In some of the FE-based works, this library is used for the implementation of FE schemes.

#### 4.2.4 Charm

Charm [6] is a framework designed for implementing various advanced cryptosystems. It is built using Python to decrease development time and code complexity while fostering component reuse.

#### 4.2.5 PBC

Pairing-based Cryptography (PBC) [38] is a C library that enables rapid development of cryptosystems based on pairings. It implements a bilinear cyclic group abstractly, hiding the programmer from mathematical details. Both PBC and Charm are used in some of the FE-based PPML implementations.

# 5 Insights

Recall that we consider the FE-based PPML works focused on training and/or inference over encrypted data.

## 5.1 Threat model

**Assumption:** In FE-based PPML, it is assumed that there is a trusted independent key management authority that is responsible for the generation of the required keys. Such assumptions are common in all FE-based works.

FE-based PPML techniques adhere to an honest but curious security model, in which both parties comply with the protocol while attempting to gain as much information as possible. In this case, the server follows the protocol but tries to learn additional information. Typically, the approach includes three primary components: a key management authority, a server, and a client. The key management authority generates encryption and decryption keys. FE requires three distinct keys, namely master public key (mpk), master secret key (msk), and functional encryption key($sk_{fe}$), that operate slightly differently than public-key cryptography techniques as discussed in section 2.

The research works in this field assume an MLaaS setting. The server holds the trained model, and the client provides the data to the server and receives the prediction results using the learned model. Additionally, in some circumstances, the capability of training models over encrypted data may be available.

The client firstly encrypts the data using the master public key before sending it to the server. In some scenarios, the client may require to preprocess the data before encryption; this varies by application. For example, computer vision applications may require scaling or transforming images prior to encryption. Normalization and standardization may be necessary in the case of structured data.

The server holds the model. The model is developed by training the neural network using client-supplied data. Because the server cannot see the data sent by the client, it must obtain the functional encryption secret key $sk_{fe}$ in order to execute functionality-based computations on the data. Either polynomial approximation uses such computations for QFE-based methods or inner-product computation in IPFE-based methods discussed in the later sections.

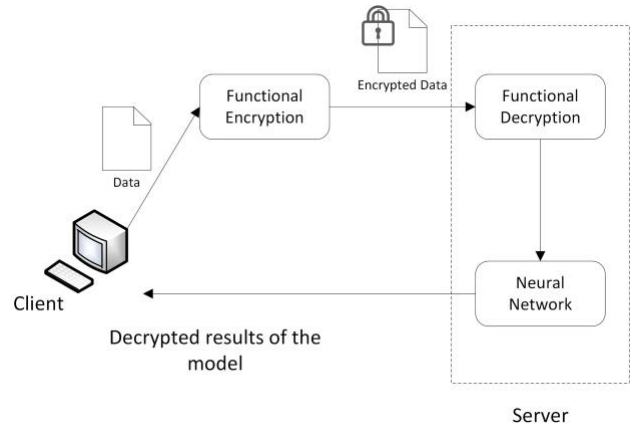


**Fig. 2.** IPFE-based PPML

## 5.2 IPFE-based PPML

Fig. 2 depicts IPFE-based PPML. Out of the available works in this field dedicated to FE-based PPML [36] and [53] use inner-product FE schemes.

#### 5.2.1 Functional encryption scheme

The work proposed in [36] and [53] use the inner product schemes proposed in [5] and [2], respectively.

#### 5.2.2 Training phase

In the approach proposed by [36], Extremely Randomized Trees are used as a machine learning model. These trees are nothing but an ensemble learning model that uses decision trees. In their work, they perform the training on plain data that is similar to regular machine learning models.

The methodology proposed in [48] uses a LeNet like 5-layer neural network. In the case of this work

**Table 2.** Comparison of FE-based PPML models in terms of functionality

| Research works | FE type | Training[1] | Prediction[2] | ML model | Security |
|---|---|---|---|---|---|
| Ligier et al. [36] | IPFE[3] | ○ | ● | ERT[4] | Selective IND-CPA[5] |
| Xu et al. [53] | IPFE | ● | ● | 5 layer NN[6] | Selective IND-CPA |
| Sans et al. [49] | QFE[7] | ○ | ● | 2 layer NN | Adaptive IND-CPA[8] |
| Ryffel et al. [47] | QFE | ○ | ● | 2 layer NN | Adaptive IND-CPA |
| Carpov et al. [17] | IPFE and QFE | ○ | ● | 2 layer NN | Adaptive IND-CPA |

[1] Training on encrypted data
[2] Prediction on encrypted data
[3] Inner-product Functional Encryption
[4] Extremely Randomized Trees
[5] Selective security against chosen-plaintext attack
[6] Neural Network
[7] Quadratic Functional Encryption
[8] Adaptive security against chosen-plaintext attack

based on IPFE, inputs are first encrypted by the client and then sent to a server. The server obtains the FE key and runs the IPFE decrypt function in order to compute the activation results of the first hidden layer. The computation between the input vector and weight matrix is done as follows:

$$A = \mathrm{ReLU}(sk_{fe}(W) * \mathrm{Encrypt}(X) + b)$$

This output obtained after the first hidden layer is then fed through the remaining layers of the neural network. So it can be said that these methodologies can work entirely on FE encrypted data. The training method over encrypted data proposed by [53] requires 57 hours to train the 5-layer neural network model. Regular neural network with the same settings on non-encrypted data takes 4 hours. This shows a massive gap between computation on encrypted data over non-encrypted data considering neural networks.

#### 5.2.3 Inference phase

The methodology proposed in [36] can perform predictions on encryption data with a prediction time of fewer than 0.1 seconds. Also, it produces a validation accuracy of 95.64 %. The work proposed in [53] can run the inference on encrypted data by following the similar use of IPFE decryption functions for the inference. Their model achieves a validation accuracy of 95.49 %.

## 5.3 QFE-based PPML

Fig. 3 depicts IPFE-based PPML. Three works use quadratic functional encryption schemes, which are defined and shown utilization towards machine learning problems in [17, 47, 49].

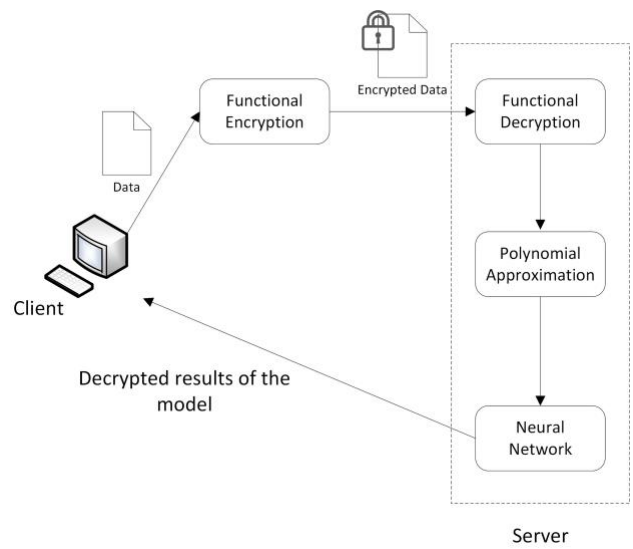


**Fig. 3.** QFE-based PPML

#### 5.3.1 Functional encryption scheme

This type of research works either use the similar or modified approach proposed by Baltico et al. [11]. Works proposed by [49] and [47] are related to each other in terms of the FE scheme and neural networks used. They proposed a novel functional encryption scheme that can be used on multivariate quadratic polynomials.

#### 5.3.2 Training phase

The training phase in this type of methodology is done on plain data similar to regular neural networks [49] and [17]. In work proposed by [47], the adversarial training approach is used in order to avoid information leakage after functional decryption. They optimized both the primary classification objective and the opposite of the collateral objective of a

particular simulated adversary simultaneously using adversarial training.

#### 5.3.3 Inference phase

In the inference phase of all the works proposed in [17, 47, 49], polynomial neural networks are used. Here they use two-layer neural networks. Wherever quadratic functional encryption is used, a special type of neural network called the polynomial neural network is used discussed in section 2.4. The approaches based on QFE have not yet been used for training the model over encrypted data because of the complexities involved in the cryptographical aspects. Also, the present functional encryption is defined for the usage for degree 2 polynomials. The methodology proposed by [49] and [47] requires fewer than 3 seconds to produce a prediction result. Moreover, in the case of QFE-based works, accuracy is shown a little higher. [49] and [47] obtain the accuracy of 97.54% and 97.7%, respectively.

## 5.4 Security

**Theorem 1** Under the decisional diffie hellman (DDH) assumption, IPFE scheme given in section 2.1 is *selectively secure against chosen-plaintext attacks* (INDIPFE-CPA).

**Theorem 2** Under the matrix decisional diffie hellman (MDDH) assumption, QFE scheme given in section 2.2 is *adaptively secure against chosen-plaintext attacks* (IND-QFE-CPA).

We defer the readers to [2] and [11, 47] for the security proofs of the theorems.

The security of the FE-based PPML approaches is dependent on the cryptographic security of the underlying FE schemes. They offer two types of security: adaptive security and selective security against chosenplaintext attacks. If the security guarantee happens only for the messages known ahead of time, it is known as selective security. However, adaptive security is a concept where a security guarantee happens even for randomly chosen messages at any point in time. The approaches proposed in [36] and [53] are selectively secure whereas the approaches proposed in [49], [47] and [41] show adaptive security based on the underlying crypto schemes.

## 5.5 Information leakage and attacks

Although functional encryption schemes provide cryptographic security to the machine learning models, there is a risk of information leakage. The outputs obtained in both the IPFE and QFE based models are in plain after the FE decryptions are done. So, if the server tries to retrieve part of the information from the obtained plain data after FE computations, there is a possibility of information leakage. [17] discusses the type of information leakage. Also, an adversarial training-based approach proposed in [47] can avoid information leakage to some extent. We believe there is a scope of attacks like model inversion attacks [24] and direct inference attacks that machine learning security researchers can study. In order to make the FE models deployable, this potential threat should be taken into consideration.

## 5.6 Implementation Details

Almost all of the methodologies use MNIST dataset [34] for the experiments. MNIST contains 70,000 grayscale images of size 28x28 pixels of handwritten single digits from 0 to 9. Out of 70K images, the training set consists of 60K images, and the test set consists of 10K images. In [17], Census Income Dataset [33] is used in addition to MNIST dataset. Census Income data was extracted from the 1994 Census bureau database. It is a multivariate dataset with 48842 instances. There are 14 attributes like age, work class, education with categorical and integer data. The task here is to predict whether a person's income exceeds $50K/yr.

*FE Libraries.* [36] implemented IPFE scheme using FLINT library [29] whereas [53] used Charm libary [6] for implementation of IPFE scheme. [49], [47] used PBC (pairing-based cryptography) [38] and [6] library for implementation of QFE scheme. [41] used GoFE libary

**Table 3.** Comparison of FE-based PPML models in terms of performance

| Research works | Year | Training time | Prediction time | FE key generation | Encryption time | Decryption | Accuracy |
|---|---|---|---|---|---|---|---|
| Ligier et al. [36] | 2017 | NA | <0.1s | 12 ms | 150 ms | 69s | 95.64% |
| Xu et al. [53] | 2018 | 57 hrs | not specified | not specified | not specified | not specified | 95.49% |
| Sans et al. [49] | 2019 | NA | a few seconds | 8ms | 8.1s | 3.3s | 97.54% |
| Ryffel et al. [47] | 2019 | NA | <3 ms | 94 ± 5ms | 12.1 ± 0.3s | 24 ± 9ms | 97.7% |
| Carpov et al. [17] | 2020 | NA | not specified | not specified | not specified | not specified | 90 % |

**Table 4.** Implementation details

| Research works | Dataset | FE Libraries | Python Libraries | System specifications |
|---|---|---|---|---|
| Ligier et al. [36] | MNIST[1] | FLINT[2] | sklearn [44] | Intel Core i7-4650U, 8GB RAM |
| Xu et al. [53] | MNIST | Charm[3] | Numpy [50] | Intel Core i7, 8GB RAM |
| Sans et al. [49] | MNIST | PBC[4]and Charm | Tensorflow [1] | Intel Core i5-6440HQ, 8GB RAM |
| Ryffel et al. [47] | MNIST | PBC and Charm | Tensorflow | Intel Core i7, 16GB RAM |
| Carpov et al. [17] | MNIST, Census Income | not specified | Keras [20] | not specified |

[1] MNIST handwritten digits dataset [34]
[2] Fast Library For Number Theory [29]
[3] A framework for rapidly prototyping cryptosystems [6]
[4] Pairing-based Cryptograph library [38]
[5] 1994 Census database [33]

proposed by [41] for the implementation of QFE scheme. We have already detailed these libraries in section 5.2.

*Python Libraries.* [36] used sklearn [44] in python for implementation of ML model. [49] and [47] implemented the ML model in Tensorflow [1] in python. [53] uses just a Numpy [50] for the implementation of ML model whereas [17] used Keras [20] in python for the implementation of ML model.

*System specifications.* Table 4 summarizes the machine specifications used for performing experiments by the various authors. The experiments done by all the research works are on intel CPUs. As CPUs are involved, there is an issue of slow computation involved during training and prediction. This is far away from the faster computations done by today's GPU-based machine learning models.

# 6 Discussion

## 6.1 Pros and Cons

*To ensure the fully privacy-preserving machine learning system, both training and prediction phases over encrypted data should be integrated into the system.* Our studies observed that for a perfect PPML system, both training and prediction should be undergone over encrypted data. By doing so, the system ensures the privacy of the user's data for both tasks. As suggested in the example of privacy-preserving cardiovascular disease prediction service, such data of the user is highly confidential. In such cases, it would be great if the cloud server can train the possessed model without seeing the user's private data. Similarly, if predictions are to be done, it would also use the encrypted inputs. So, this interesting concept of training and prediction without seeing helps us develop a fully privacy-preserving machine learning system. Presently, only [53] focuses on both of these tasks, but it is not efficient in terms of time.

*Considering information leakage and adaptive security simultaneously is highly important.* Security of the FE-based PPML schemes is entirely dependant on the underlying FE scheme. IPFE is protected by selective security, whereas QFE is protected by selective as well as adaptive security against chosen-plaintext attack. There is a theoretical advancement by [4] towards making IPFE adaptively secured, but FE-based PPML works do not yet implement it. Also, information leakage proposed by [17] is the only work that discusses this concept. This work also follows the traditional selective IPFE and adaptive QFE schemes. This raises a high need to consider both information leakage and adaptive security while building an FE-based PPML scheme.

*There is no fit-for-all methodology.* None of the existing solutions considers all the criteria for a perfect PPML system. However, we can rank the methodologies from each group i.e one from the IPFE-based scheme and another from the QFE-based scheme. In the IPFEbased scheme, [53] satisfies most of the criteria, whereas [47] performs the best in QFE-based methodologies. If prediction accuracy and privacy is considered, then [47] leads, whereas the overall performance of the system in terms of security, privacy, and accuracy is concerned, then [53] leads over others.

## 6.2 Challenges and future research directions

### 6.2.1 Functionality enhancement in FE scheme

The current research done on FE is limited to two functionality on integers: inner product and bilinear maps. Recall that functionality is a function computed over encrypted data to obtain decrypted results. As discussed, inner product functionality is used in the IPFE scheme, whereas bilinear maps are used in the QFE scheme. Because of this limited availability of functionalities, FEbased PPML methodologies still require enhancement. For example, QFE-based machine learning methods can use only 2-degree polynomial networks. Also, FE cannot perform Min/max and comparison operations. If enhancement in FE functionality is done, it will be helpful for PPML researchers to come up with practical solutions for supervised and unsupervised machine learning problems. However, these systems are not at a level to be used in real-world applications.

### 6.2.2 Improving efficiency

Though FE-based PPML solutions have shown promising results on partially encrypted machine learning, there is a need to improve the efficiency. Ciphertexts and keys generated with larger security parameters are slower compared to other cryptographic approaches. This becomes a threat when computations are performed on large-scale datasets. Therefore this efficiency stands as a big challenge in FE-based PPML methodologies.

### 6.2.3 Improvement in Security and Privacy of FE scheme

As discussed in the previous subsection, the FE-based PPML methodologies' security depends on the underlying FE scheme. IPFE schemes [2, 5] are selectively secured against chosen-plaintext attacks under the decisional-diffie hellman (DDH) assumption. QFE schemes [11, 47] are adaptively secured against chosen plaintext attacks under Matrix Decisional DiffieHellman assumption. These FE schemes will lag in stricter security applications such as the defense domain. Information leakage in PPML based FE schemes needs to be explored and improved. In addition to this, machine learning attacks are also interesting to consider in this area.

### 6.2.4 Enhancing Privacy-preserving Neural Networks.

Today's FE-based works have shown their demonstration only up to 5-layer neural network. There is a scope to enhance the structure of neural networks. As discussed above, QFE-based methods are limited to using only degree-2 polynomial networks. So, the enhancement of these networks is somehow dependent on the underlying FE scheme. Implementation of various activation functions is also dependent on the underlying FE scheme. Apart from this, complex deep learning convolutional neural networks like VGGNet, AlexNet, and GoogleNet are far from implementing FE. Also, the methods like transfer learning can be applied if there exists a functionality that can decrypt the encrypted model and retrieve the saved parameters like general CNNs.

### 6.2.5 Training-centric PPML systems

As the PPML is growing day by day, training over encrypted data is gaining the attention of researchers [51]. Presently, FE-based PPML schemes proposed in the literature are more inference-centric. Although [53] has successfully proposed a way for training a neural network using functional encryption, it lacks computational efficiency. There is a high need for PPML systems that can perform both training and inference over encrypted data.

### 6.2.6 Training from multiple data sources

As far as machine learning is concerned, there are methodologies that focus on training models from multiple data sources in literature [51]. The enhanced versions of FE scheme like multi-input functional encryption [3, 16] can be used to serve this purpose. These schemes facilitate using multiple vectors to perform computation based on inner-product functionality. Such schemes could leverage the problem of training the model possessed by a cloud server for doing training from multiple data sources.

### 6.2.7 Using Multi-authority and Decentralized extensions of FE

The FE scheme used in PPML services requires a trusted third party or trusted authority to be involved in generating the keys. The scheme proposed by [7] can be used to avoid the involvement of trusted authorities. In addition to this, there is a great scope for making the FE schemes used in PPML decentralized by using the approaches proposed by [22] and [21].

### 6.2.8 Need for open source library support

Functional encryption has gained widespread attention from researchers as far as the theoretical aspects are concerned. However, it still lacks practical library implementations compared to its predecessor, homomorphic encryption. For example, HElib [28] by IBM research group, and SEAL [18] by Microsoft research group are great tools for performing the homomorphic encryptionrelated task. Presently, commendable efforts have been taken by FENTEC group [41] for providing C and Go language versions of FE libraries, namely CiFEr and GoFE. These libraries are in the development stage and are not yet ready to be deployed in

production. So, there is a considerable need for practically applicable functional encryption libraries.

#### 6.2.9 Need for hardware acceleration support

In all the implementations of FE-based PPML methodologies studied, computations are done on the CPU. Until now, there has been no GPU support available for FE implementation. Because of these limitations, we lack the opportunity to utilize the highest GPU compute available today. Real-world applications in machine learning are trained on massive data. So performing computations in such applications is very timeconsuming if done on CPU. The work proposed by [10] uses SoC implementation to provide acceleration support, but it needs special hardware. If easy-to-use hardware acceleration support using widespread available GPU or FPGA is provided in such applications, it will speed up the computations faster. Such acceleration will eventually help in making FE deployable in real-world applications.

#### 6.2.10 Improving scalability and application to non-image datasets

Present works focused on FE-based PPML are limited to using MNIST like small datasets for their experiments, and there are no implementations for larger datasets. In addition to this, the applications to nonimage datasets like text corpus may also be explored. Therefore, this stands as a future research direction in improving FE utilization in real-world applications.

## 7. Conclusion

Privacy-preserving machine learning has gained widespread attention among industry and academic researchers in recent years. Although approaches based on fully homomorphic encryption, secure multi-party computation, and differential privacy have been extensively studied, functional encryption-based solutions are still less investigated. In this paper, we provide a summary and systematization of privacy-preserving machine learning approaches based on functional encryption. Our analysis assessed the extent to which previous work has addressed the PPML using FE and identified key weaknesses in this area. Additionally, our analysis demonstrates that FE-based approaches could significantly contribute to the PPML issues, but there exist challenges that should be addressed to achieve practical solutions. We hope that this effort paves the path for the research community to investigate this emerging yet important topic.

## References

1. Iwai, S., & Sikdar, B. (2023). "Functional Encryption in Cloud-Based Privacy-Preserving Neural Networks." IEEE Access, 11, 34127-34139.
2. Liu, Y., & Wen, Y. (2023). "Privacy-Preserving Machine Learning Based on Cryptography." IEEE Security & Privacy, 21(1), 34-45.
3. Xu, R. (2020). "Functional Encryption Based Approaches for Practical Privacy-Preserving Machine Learning." University of Pittsburgh Dissertation.
4. Zhou, M., & Shi, E. (2022). "The Power of the Differentially Oblivious Shuffle in Distributed Privacy Mechanisms." CryptoPPML Workshop.
5. Xue, R., et al. (2023). "A Review of Secure Multi-Party Computation in Privacy-Preserving ML." ACM Computing Surveys, 56(6).
6. Ligier, T., & Marc, H. (2021). "Inner Product Functional Encryption for Secure Classification." Proceedings of the IEEE Symposium on Security and Privacy.
7. Panzade, P., & Takabi, D. (2022). "Privacy Preserving Machine Learning using Functional Encryption: Opportunities and Challenges." arXiv preprint arXiv:2204.05136.
8. Kim, J., & Park, C. (2023). "Differentially Private Federated Learning for Healthcare Applications." Journal of Biomedical Informatics, 134, 104800.
9. Ryffel, T., et al. (2019). "QFE with Adversarial Training for Privacy-Preserving Prediction." PETS Symposium.
10. Zhang, K., & Wang, Q. (2022). "Homomorphic and Functional Encryption for ML Confidentiality." ACM Transactions on Privacy and Security, 25(3).
11. Marc, H., et al. (2020). "FE Libraries and Applications for ML Privacy." IEEE Transaction on Information Forensics and Security, 15(2), 392-406.
12. Li, Z., & He, X. (2023). "Performance Evaluation of FE Libraries for Data Science." Data & Knowledge Engineering, 146, 102304.
13. Sinha, S., et al. (2021). "Comparing FHE and FE: Encrypted Learning in Practice." Journal of Data Privacy, 14(2), 96-111.
14. Chatterjee, A., et al. (2022). "Functional Encryption for Decentralized ML Systems." IEEE Internet of Things Journal, 9(12), 7915-7928.
15. Ahmed, I., et al. (2023). "Cryptography Techniques for Privacy in Machine Learning." Computer Security, 111, 102396.
16. Nair, V., & Giridharan, S. (2022). "Secure Federated Analytics with Functional Encryption." Security and Communication Networks, 2022, 3892638.
17. Lu, S., & Fan, J. (2022). "Design Challenges for FE-Enabled Healthcare AI." Health Informatics Journal, 28(6), 104912.
18. Dolev, S., et al. (2023). "Adversarial Robustness in FE Privacy Solutions." Data Security Advances, 12(2), 98-117.
19. Sharma, V., & Chen, P. (2022). "Benchmarking FE in Edge AI Applications." Edge Computing, 5(4), 150-162.
20. Wang, J., & Huang, R. (2020). "Usability Tests for Functional Encryption APIs." Software Security Reports, 88(2), 41-66.
21. Asokan, K., et al. (2020). "Distributed Training Protocols Leveraging FE." Machine Learning and Security Journal, 3(7), 101-111.
22. Du, Y., & Lin, C. (2023). "Differential Privacy and Functional Encryption for ML Pipelines." Infocom 2023 Proceedings.
23. Bianchi, L., Roy, D. (2021). "Expressiveness versus Efficiency in Functional Encryption." Cryptologia, 45(3), 314-328.
24. Garcia, A., et al. (2022). "Survey of FE Approaches in Secure Collaborative Learning." Proceedings of Privacy-Preserving Data Analysis, 19-30.

25. Takahashi, M., & Mori, S. (2023). "Privacy Guarantees for FE-based ML Models." Health Informatics Review, 29(3), 91-111.
26. Dewar, E., & Chou, T. (2022). "Cryptographic Primitives in Modern ML Privacy." Cryptography Today, 19(6), 59-78.
27. Zhang, Y., & Qiu, L. (2021). "Functional Encryption in Federated Learning: Survey and Perspectives." Security Engineering Research, 6(4), 201-215.
28. Leon, C., et al. (2023). "Trends and Prospective of FE for PPML." Privacy Engineering Symposium, 10(1), 13-27.
29. Dong, Q., et al. (2020). "Efficient FE Protocols for ML Applications." AI & Security Insights, 4(2), 67-80.
30. Stearns, M., & Zhao, P. (2022). "Libraries and Toolkits for Functional Encryption." Software Security Advances, 91(4), 19-35.
31. Li, M., & Singh, V. (2021). "Homomorphic vs. Functional Encryption in ML Privacy." Cryptography and Communications, 14(5), 435-456.
32. Ornelas, F., & Wada, N. (2023). "Cross-domain Applications of FE-based PPML." Computer Security Review, 21(1), 61-74.
33. I. Chillotti, N. Gama, M. Georgieva, and M. Izabachene. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *international conference on the theory and application of cryptology and information security*, pages 3–33. Springer, 2016.
34. F. Chollet et al. Keras: The python deep learning library. *Astrophysics Source Code Library*, pages ascl–1806, 2018.
35. J. Chotard, E. Dufour-Sans, R. Gay, D. H. Phan, and D. Pointcheval. Dynamic decentralized functional encryption. In *Annual International Cryptology Conference*, pages 747–775. Springer, 2020.
36. C. Dwork, F. McSherry, K. Nissim, and A. Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*, pages 265–284. Springer, 2006.
37. Mehta, D. B. (2023). Privacy-Preserving Machine Learning Architecture for Cross-Channel Advertising Optimization in a Large-Scale Social Media Platform. International Journal of Communication Networks and Information Security (IJCNIS), 15(8), 23–32. https://www.ijcnis.org/index.php/ijcnis/article/view/8396
38. M. Fredrikson, S. Jha, and T. Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 1322–1333, 2015.
39. C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009. [26] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International Conference on Machine Learning*, pages 201–210, 2016.